

1/6

From Jupyter notebooks to workflow graphs

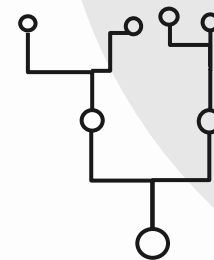
Reina Al Masri

Supervised by Sarah Cohen-Boulakia and George Marchment

June 2026

L1SN
LABORATOIRE INTERDISCIPLINAIRE
DES SCIENCES DU NUMÉRIQUE

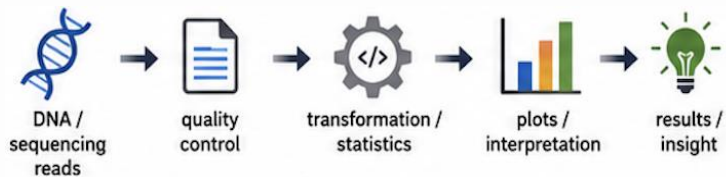
université
PARIS-SACLAY



Context

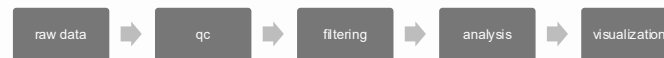
2/6

Bioinformatics analysis



Bioinformatics uses code to analyse biological data through multiple connected steps

What is a workflow?



A workflow = an explicit chain of dependent analysis steps



Formal Workflow Systems



- Systems such as **Nextflow** make dependencies implicit
- **Bioflow-Insight** can statically reconstruct graph views, which **MetroFlow** displays as metro maps, making these dependencies explicit.



Problem

3/6

Jupyter notebooks are widely used



Jupyter notebooks combine text, plots and outputs in one interactive document



9.7M Public Jupyter notebooks downloaded from Github in 2020

JetBrains/Datalore report: 9.72M notebooks in Oct 2020 vs 1.23M two years earlier. Source: JetBrains Blog (2020)



35% of data professionals use Jupyter notebooks

Over 40% of these users spend more than 20% of their working time in them Source: JetBrains Survey (2023)

Motivation : The more notebooks are used for scientific analyses, the more important it becomes to understand, visualize, and reuse them.

Graphs provide a way to make hidden dependencies explicit.

The notebook problem

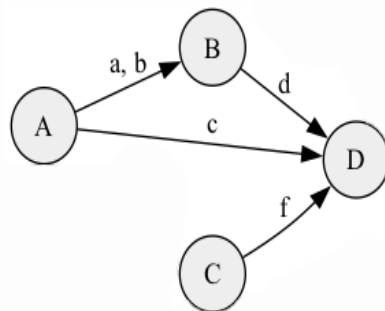
```
[ ]: # Cell A
a = 1
b = 1
c = 1
```

```
[ ]: # Cell B
d = a + b
```

```
[ ]: # Cell C
f = 2
```

```
[ ]: # Cell D
res = c + d + f
```

Static
analysis



- Notebook order does not fully reveal computational structure
- A notebook cell may contain an entire analysis phase

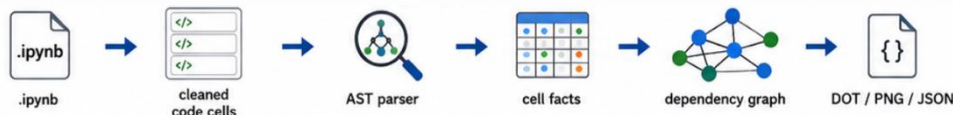


Variables and functions may be defined in one cell and used much later

Materials and methods

4/6

Prototype pipeline



Contribution: a static reconstruction of useful dataflow information from one notebook at a time, supporting visualization, comprehension, and integration with the BioFlow-Insight graph API.

Python's AST module

AST = code as a tree

Each node is one syntactic element:
Assignment, name, operation,
function, call, etc.

Example

`b = a + 1` →



Load = read
Store = defined



Results and analysis

5/6

Output types

```
[ ]: #Cell A
import random
def fun(x):
    return x+random.random()
```

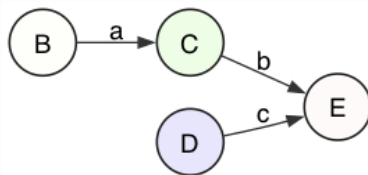
```
[ ]: #Cell B
a = 1
```

```
[ ]: #Cell C
b = fun(a)
```

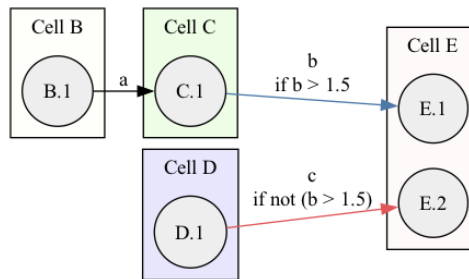
```
[ ]: #Cell D
c = 1
```

```
[ ]: #Cell E
if(b>1.5):
    res = fun(b)
else:
    res = fun(c)
```

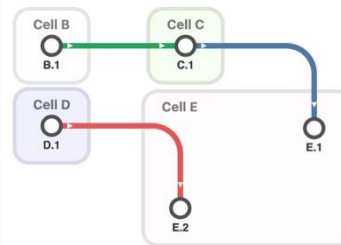
Cell-level graph



Statement-level graph
(expanded graph)



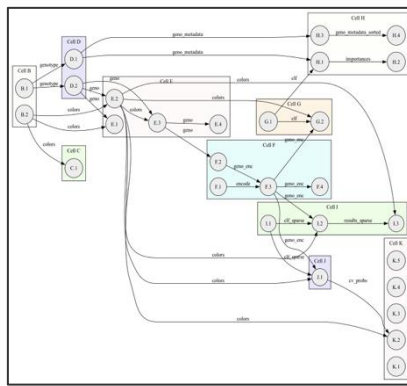
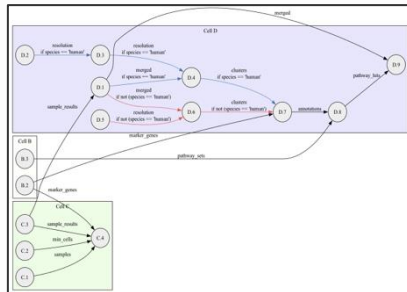
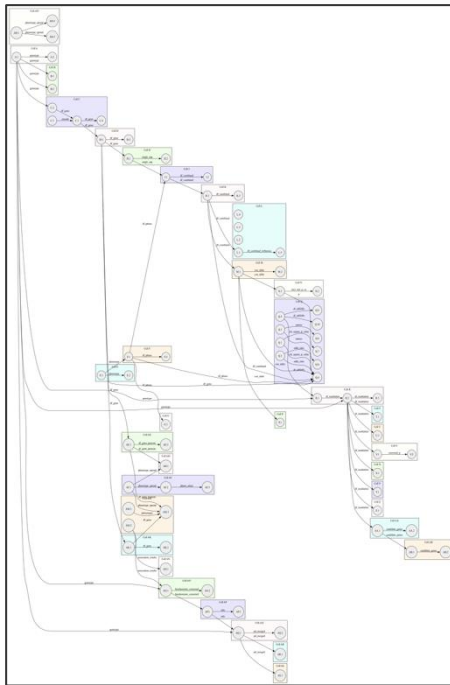
Metro map



Modeling rules

- Flat graph: cell = node
- Expanded graph: cell = box, statement = node
- $A \rightarrow B$ means B reads x, A last defined x
- Imports and function name alone do not create nodes
- function uses external x, $\rightarrow$ points to the call cell

Conclusion



6/6

Successfully implemented a parser that reconstructs dataflow from Jupyter notebooks, turning notebook code into cell-level and statement-level graphs, making hidden dependencies visible.

This extends the BioFlow-Insight approach from formal Nextflow workflows to widely used Jupyter notebooks.

What's next?

- Improve graph labels
- Handle more control flow: loops, **if** inside loops, **try/except**, **match**, **with**
- Go beyond one notebook: files, imports, other notebooks
- Add runtime information: execution order, hidden state, dynamic values